CPU Core Counter

This package is a tiny utility to get the number of CPU cores.

```
composer require fidry/cpu-core-counter
```

Usage

```php
use Fidry\CpuCoreCounter\CpuCoreCounter;
use Fidry\CpuCoreCounter\NumberOfCpuCoreNotFound;
use Fidry\CpuCoreCounter\Finder\DummyCpuCoreFinder;

$counter = new CpuCoreCounter();

// For knowing the number of cores you can use for launching parallel processes:
$counter->getAvailableForParallelisation()->availableCpus;

// Get the number of CPU cores (by default it will use the logical cores count):
try {
    $counter->getCount();   // e.g. 8
} catch (NumberOfCpuCoreNotFound) {
    return 1;   // Fallback value
}

// An alternative form where we not want to catch the exception:

$counter = new CpuCoreCounter([
    ...CpuCoreCounter::getDefaultFinders(),
    new DummyCpuCoreFinder(1),  // Fallback value
]);

// A type-safe alternative form:
$counter->getCountWithFallback(1);

// Note that the result is memoized.
$counter->getCount();   // e.g. 8
```

Advanced usage

Changing the finders

When creating `CpuCoreCounter`, you may want to change the order of the finders used or disable a specific finder. You can easily do so by passing the finders you want

```php
// Remove WindowsWmicFinder
$finders = array_filter(
```

```
    CpuCoreCounter::getDefaultFinders(),
    static fn (CpuCoreFinder $finder) => !($finder instanceof WindowsWmicFinder)
);

$cores = (new CpuCoreCounter($finders))->getCount();

// Use CPUInfo first & don't use Nproc
$finders = [
    new CpuInfoFinder(),
    new WindowsWmicFinder(),
    new HwLogicalFinder(),
];

$cores = (new CpuCoreCounter($finders))->getCount();
```

Choosing only logical or physical finders

`FinderRegistry` provides two helpful entries:

- `::getDefaultLogicalFinders()`: gives an ordered list of finders that will look for the *logical* CPU cores count.
- `::getDefaultPhysicalFinders()`: gives an ordered list of finders that will look for the *physical* CPU cores count.

By default, when using `CpuCoreCounter`, it will use the logical finders since it is more likely what you are looking for and is what is used by PHP source to build the PHP binary.

Checks what finders find what on your system

You have three scrips available that provides insight about what the finders can find:

```
# Checks what each given finder will find on your system with details about the
# information it had.
make diagnose                                          # From this repository
./vendor/fidry/cpu-core-counter/bin/diagnose.php  # From the library
```

And:

```
# Execute all finders and display the result they found.
make execute                                        # From this repository
./vendor/fidry/cpu-core-counter/bin/execute.php  # From the library
```

Debug the results found

You have 3 methods available to help you find out what happened:

1. If you are using the default configuration of finder registries, you can check the previous section which will provide plenty of information.

2. If what you are interested in is how many CPU cores were found, you can use the `CpuCoreCounter::trace()` method.
3. If what you are interested in is how the calculation of CPU cores available for parallelisation was done, you can inspect the values of `ParallelisationResult` returned by `CpuCoreCounter::getAvailableForParallelisation()`.

Backward Compatibility Promise (BCP)

The policy is for the major part following the same as Symfony's one. Note that the code marked as `@private` or `@internal` are excluded from the BCP.

The following elements are also excluded:

- The `diagnose` and `execute` commands: those are for debugging/inspection purposes only
- `FinderRegistry::get*Finders()`: new finders may be added or the order of finders changed at any time

License

This package is licensed using the MIT License.

Please have a look at `LICENSE.md`.